

s5.01 — PKI and KMS mechanism reference

Supporting reference s5.01 in [the bank](#); not a ranked question. Feeds s2.01–s2.06.

Scope: 40-minute read. Part A/B = concepts. Part C = design doc for an internal CA service (MVP0/MVP1).

Where a number is not from a cited doc it is marked (assume) or order of magnitude.

JD tie-in ([../0.req/oracle senior ic3.pdf](#)): PKI team — certificate lifecycle, key management, trust services, multi-tenant, backward compatibility. This file is the **vocabulary + mechanism** layer; the interview-format design of the same domain is [s2.01](#), the OCI product mapping is [../4.1o-arch.md §3/§5](#).

Tags: [hot path] · [async] · [control plane]. ? = likely follow-up + the answer to give. 🗣️ = say verbatim.

Index

- [Part A — What a PKI is](#)
 - A.1 The problem PKI solves
 - A.2 The objects: key pair, certificate, CA, chain, trust store, revocation
 - A.3 Public key vs. certificate — the relation
 - A.4 What the Oracle doc you linked actually describes
 - A.5 Anatomy of an X.509 v3 certificate + how a verifier walks it
 - A.6 Revocation options compared (CRL / OCSP / stapling / short-lived)
 - A.7 Algorithms, file formats, and the openssl/keytool one-liners
 - [Part B — KMS vs. PKI vs. Secrets \(AWS KMS / OCI Vault\)](#)
 - B.1 Taxonomy: four different services people lump together
 - B.2 AWS KMS features
 - B.3 Rotation — what actually rotates
 - B.4 Access: role → authenticated → authorized → key used (exact sequence)
 - B.5 OCI equivalents (Vault, Certificates, IAM)
 - B.6 KMS extras: key states, grants, DEK caching, Sign for a CA
 - [Part C — Design doc: internal Certificate Authority service](#)
 - C.0 One-page design doc
 - C.1 Requirements
 - C.2 High-level design
 - C.3 APIs
 - C.4 Data model
 - C.5 Core flows (numbered sequences)
 - C.6 Failure modes
 - C.7 MVP ladder + what we evolve next
 - C.8 Key function code: `issue(csr)` — Python, then Java with a KMS-backed signer
 - C.9 Build vs buy: managed CAs, Vault PKI, step-ca, SPIRE, cert-manager
 - C.10 ? Pop-up questions (L4 answer + L5 extension)
 - 🗣️ [One-liners for the interview](#)
 - [References](#)
-

Part A — What a PKI is

A.1 The problem PKI solves

Alice — "I am api.bank.com, here is my public key" —> Bob

▲
How does Bob know the key really belongs to api.bank.com
and not to a man-in-the-middle?

Answer: a third party Bob already trusts (CA) SIGNED the binding
"this public key ↔ this name". That signed binding = certificate.

- Intuition: asymmetric crypto lets anyone verify a signature or encrypt to you with only your **public key**. It does *not* tell anyone whose key it is.
 - PKI is the machinery that binds a public key to an identity and lets strangers check that binding without a shared secret.
 - Without it, every pair of parties needs an out-of-band key exchange (what Kerberos/shared-secret systems do — fine inside one org, useless on the open internet).
- Defeater: PKI only moves trust, it does not create it.
 - You still have to trust the root CA. Compromise the root and every cert it issued is a lie (DigiNotar 2011, public postmortem — see References).

A.2 The objects

Root CA (offline, self-signed) — signs —> Intermediate CA (online) — signs —> Leaf cert (server / client / code)

in TRUST STORE of clients in the "CA" store / chain file presented in TLS

handshake revocation lists

CRL / OCSP responder

Object	What it is	What it is not
Key pair	Private key (kept secret, ideally in an HSM) + public key (mathematically derived from it)	Not an identity. Just numbers.
CSR (certificate signing request)	Your public key + the name you claim + a signature with your private key (proves you hold it)	Not a cert. A CA has not vouched yet.
Certificate (X.509)	{subject name, public key, validity window, key usage, extensions, issuer name} signed by the issuer's private key	Not a secret. Freely distributable.
CA (certificate authority)	A key pair whose public key is pre-installed as trusted, plus policy for what it will sign	Not magic. Just a signer with a reputation.
Chain	Leaf → intermediate(s) → root. Verifier walks up until it hits a root in its trust store	Order matters; a missing intermediate is the #1 "works on my machine" TLS bug.
Trust store	The set of root certs a verifier accepts (OS store, JVM cacerts, Oracle Wallet)	Not global. Each verifier has its own.
Revocation	CRL (signed list of revoked serials) or OCSP (ask the CA "is serial X still good?")	Not instant. Clients cache; many browsers soft-fail. Short-lived certs are the modern substitute.

- Intuition: the whole system is "signatures over signatures" ending at a key you installed by hand.
 - Mechanism: verifier checks sig(leaf) with intermediate.pub, then sig(intermediate) with root.pub, then "is root.pub in my trust store?", then name match, validity window, key-usage bits, revocation.
 - Defeater: every one of those checks is a separate bug class.
 - ◆ Name check skipped → any valid cert works (classic library bug).

- ◆ Validity clock skew → outages at midnight cert expiry.
- ◆ Revocation soft-fail → a revoked cert still accepted when OCSP is unreachable.

A.3 Public key vs. certificate — the relation

```

private key (secret) —derive→ public key (public, anonymous) —wrap in X.509 + CA signature→ certificate (public, NAMED, TRUSTED-by-issuer, EXPIRES)

```

- A certificate **contains** a public key. It adds three things the bare key lacks:
 - identity (subject / SANs),
 - a trusted third party's signature over that identity binding,
 - lifecycle (not-before / not-after, revocation pointer).
- The private key is **never** in the certificate. If you see a file with both, it is a bundle (PKCS#12 .p12 / .pfx, Oracle Wallet ewallet.p12), not a certificate.
- Same public key can appear in many certs (re-issued, different names).
 - Rotating the *certificate* does not rotate the *key*; rotating the *key* forces a new cert.
 - ☛ "A certificate is a public key plus a name plus a signature plus an expiry. The private key never leaves."

A.4 What the Oracle doc you linked actually describes

```

sqlnet.ora
WALLET_LOCATN = (SOURCE=(METHOD=FILE)(METHOD_DATA=(DIRECTORY=...))) ← Oracle Wallet (ewallet.p12)
WALLET_LOCATN = (SOURCE=(METHOD=MCS)) ← Microsoft Certificate Store
      |
      v
Oracle Net TCPS (TLS) uses certs + trust points from whichever store is selected

```

- It is *not* a cloud KMS. It is Oracle **Database** client/server PKI plumbing on Windows.
 - (Oracle docs, 18c) Oracle PKI has two components: Oracle Wallets and Oracle Wallet Manager (OWM). Wallets store digital certificates, trust points, and private keys used for encryption, decryption, signing and verification.
 - (Oracle docs, 18c) Consumers: Enterprise Security Manager, LDAP-enabled Enterprise Manager, Oracle SSL authentication, Oracle Database, WebLogic.
- The "Windows PKI" half is about substituting the OS store for the wallet:
 - (Oracle docs, 18c) Windows keeps certs and CRLs in logical stores (pointers) over physical stores; the standard ones are MY/Personal (certs whose private key is available), CA (issuing/intermediate CAs), and ROOT (self-signed trusted roots).
 - (Oracle docs, 18c) Microsoft Certificate Services has three modules: Server Engine (handles all requests), Intermediary (accepts new-cert requests from clients and submits them), and Policy (the rules controlling issuance, customizable).
 - ◆ That three-module split — request intake / issuance engine / policy — is exactly the shape of the CA we design in Part C.
 - (Oracle docs, 18c) Selecting the store: set WALLET_LOCATN = (SOURCE=(METHOD=MCS)) in sqlnet.ora; Oracle then uses TCPS with X.509 certs and trust points from the user's Microsoft store for SSL authentication.
 - Gotchas the doc itself flags: (Oracle docs, 18c) only certs created with the Microsoft Enhanced Cryptographic Provider work, and if several certs match a key usage the first one retrieved wins. (Oracle docs, 18c) Windows rejects SSL keys shorter than 1024 bits.
- So in one sentence: the Oracle page is about **where the client finds its certs and private key** for a TLS DB connection — a trust-store/key-store question — not about issuing or managing keys.

A.5 Anatomy of an X.509 v3 certificate + how a verifier walks it

```
Certificate ::= SEQUENCE {
  tbsCertificate      ← "to be signed": everything the CA vouches for
    version v3, serialNumber, signature (alg id), issuer (DN),
    validity {notBefore, notAfter}, subject (DN), subjectPublicKeyInfo,
    extensions [3]      ← where the real semantics live (below)
  signatureAlgorithm  ← e.g. ecdsa-with-SHA256
  signatureValue       ← issuer's signature over DER(tbsCertificate)
}
```

Extension	What it says	Why it matters / defeater
Subject Alternative Name (SAN)	DNS / IP / URI (spiffe://...) / email names the cert is valid for	The name check uses SAN, not CN (RFC 6125; Chrome dropped CN fallback in v58, 2017). A cert with only a CN fails in modern clients.
Key Usage (KU)	digitalSignature, keyEncipherment, keyCertSign, cRLSign	A CA cert must carry <code>keyCertSign</code> ; a CRL signer must carry <code>cRLSign</code> . Marked critical → verifier must honor it.
Extended Key Usage (EKU)	serverAuth, clientAuth, codeSigning, OCSPSigning	mTLS leaf needs both <code>serverAuth</code> and <code>clientAuth</code> ; forgetting <code>clientAuth</code> fails only when the service acts as a client — the bug shows up one hop away.
Basic Constraints	CA:TRUE/FALSE + pathLenConstraint	Intermediate: CA:TRUE, pathLen=0 → it cannot mint sub-CAs. A leaf with CA:TRUE is a mis-issuance that lets any service become a CA (canonical pattern of old CA bugs).
Name Constraints (on CA certs)	permitted / excluded subtrees: DNS:.prod.internal, URI:spiffe://prod/	Bounds blast radius of a stolen intermediate: it can only sign names in its subtree. Defeater: older verifiers ignore it → defense-in-depth, never the only control.
SKI / AKI (subject / authority key id)	Hash of the public key, on the cert and on its issuer respectively	How path building links leaf → issuer when two intermediates share a DN (exactly the rotation-overlap case in C.5). Mismatched AKI → verifier picks the wrong intermediate → "unknown issuer".
CRL Distribution Points / AIA	URLs for the CRL, the OCSP responder, and the issuer cert (caIssuers)	Browsers fetch a missing intermediate via AIA; Java (<code>com.sun.security.enableAIAcaIssuers=false</code> by default) and OpenSSL do not → "works in Chrome, fails in the JVM" is almost always a chain-file problem.
critical flag	Per extension: reject the cert if you do not understand this extension	Non-critical unknown extensions are ignored. Mark KU/BasicConstraints critical; SAN critical only when the subject DN is empty.

- Verifier algorithm (RFC 5280 path validation) — every step is its own bug class:
 - 1. Build the path: leaf → issuer by AKI/SKI + issuer name, up to a cert in the trust store (AIA fetch if the verifier supports it).
 - 2. For each cert: signature checks against the parent's public key; `notBefore ≤ now ≤ notAfter`; CA:TRUE and pathLen on every non-leaf; name constraints of every ancestor applied to the leaf's SANs; KU/EKU compatible with the use.
 - 3. Name match: requested host / expected SPIFFE ID ∈ leaf SANs (wildcards only one label deep).
 - 4. Revocation: CRL / OCSP / staple per policy (A.6).
 - Java: chain = `X509TrustManager.checkServerTrusted` (PKIX `CertPathValidator`); hostname = separate step, on by default in `HttpsURLConnection`, **off** on a raw `SSLSocket` unless `SSLParameters.setEndpointIdentificationAlgorithm("HTTPS")`.
 - ◆ Defeater: a custom `TrustManager` that returns without throwing "to make it work" disables step 2 entirely — the most common Java TLS vulnerability.
 - ☛ "Trust is decided by four independent checks — signature chain, validity, name, revocation — and each one has its own famous failure."

A.6 Revocation options compared

Option	What it is	Strength	Weakness	Choose when
CRL	CA-signed list of revoked serials with <code>thisUpdate / nextUpdate</code> , fetched from the CDP URL	Offline-verifiable, cacheable, CDN-servable; no per-handshake dependency	Staleness = publish interval; grows with revocations; many clients never fetch it	Internal PKI, few revocations, verifiers that cannot do online checks (DBs, LBs, appliances)
OCSP	Verifier asks the responder "is serial X good?" and gets a short-lived signed answer	Fresh (minutes); small answer	Responder = availability + latency dependency on the hot path; privacy leak (CA sees who talks to whom); soft-fail in practice (Chrome disabled live OCSP in 2012)	You need minute-level revocation on long-lived certs and control the verifiers
OCSP stapling (+ must-staple)	Server fetches its own OCSP response and attaches it in the handshake; <code>must-staple</code> ext makes a missing staple a hard fail	No client→CA call, no privacy leak, hard-fail is possible	Server must refresh the staple; must-staple support is patchy; still needs the responder up for the server	Public-facing TLS servers
Short-lived certs	TTL of hours–days; no revocation at all, expiry is the revocation	No revocation infra on the verifier path; compromise window $\leq$ TTL	Issuance becomes a hot path with an availability SLO; clock skew matters more; renewal must be automated at every consumer	Internal mTLS with automated identity (SPIFFE); public web is moving here (Let's Encrypt 6-day certs, 2025)
Push-based filters (CRLite / CRLSets)	Browser vendor pushes a compact set/filter of all revoked certs	Fresh and no per-site fetch	Only works when one party controls the verifier fleet	Browser vendors; mention only

- Pick for Part C: **short-lived leaves + CRL as the backstop** for the intermediate and for any long-lived legacy cert.
 - Deciding variable: can every consumer renew automatically? If not (Oracle wallets on DB hosts, appliances), that consumer gets a longer cert **and** a CRL it actually fetches.
 - Defeater: soft-fail. If the verifier accepts on "revocation unreachable", revocation is theater; hard-fail turns a CRL outage into a global outage. Short TTLs are the way out of that dilemma.

A.7 Algorithms, file formats, and the one-liners

Algorithm	Key / signature size	Speed (order of magnitude)	Use / defeater
RSA-2048 / 3072 / 4096	256 / 384 / 512-byte signature	Verify fast; sign ~1 ms for 2048, 4096 several × slower	Universal compatibility (old Windows, Oracle wallets). Big certs → bigger handshakes. 2048 is the floor; 4096 for a root that lives 20 y.
ECDSA P-256 / P-384	64 / 96-byte signature; cert ~4× smaller than RSA-2048	Sign fast, verify fast	TLS 1.3 default; KMS/Vault support (ECC_NIST_P256/384). Defeater: nonce reuse leaks the key (Sony PS3, 2010) → use RFC 6979 deterministic nonces or an HSM.
Ed25519	64-byte signature	Fastest, deterministic by design	Not in AWS KMS as of writing (check current spec list); patchy TLS/HSM/Windows support → wrong choice for a CA that must serve Oracle/Windows clients.
Hash	SHA-256 minimum		SHA-1 signatures are forgeable (SHattered, 2017) and rejected by all modern verifiers.

- Chains may mix families in X.509 (RSA root signing an ECDSA intermediate is legal); **OCI Certificates does not allow mixing** inside one chain (`../4.10-arch.md §3`) — say which rule you are quoting.

Format	What it is	Note
PEM	Base64 DER between ----- BEGIN ...----- lines	Concatenable → the "chain file" = leaf + intermediates, leaf first
DER	Raw binary ASN.1	What is actually signed; PEM is just its transport
PKCS#10	CSR	Public key + subject + requested extensions + self-signature (proof of possession)
PKCS#8	Private key container	Optionally password-encrypted; the modern replacement for BEGIN RSA PRIVATE KEY
PKCS#12 (.p12/.pfx)	Private key + cert chain in one encrypted bundle	Java keystore default since Java 9; Oracle Wallet ewallet.p12 is one (cwallet.sso = auto-login copy)
JKS	Legacy Java keystore	Still what many -Djavax.net.ssl.trustStore flags point at; convert with keytool -importkeystore
PKCS#7 (.p7b)	Cert bundle, no keys	How Windows/CA UIs hand you a chain
PKCS#11	API to an HSM/smart-card, not a file	How software talks to an HSM without ever holding the key; cloud KMS is the network version

```

openssl x509 -in leaf.pem -noout -text                # read every field/extension above
openssl verify -CAfile root.pem -untrusted int.pem leaf.pem  # chain build + validity +
BasicConstraints
openssl s_client -connect host:443 -servername host -showcerts  # what the server actually sends
(missing intermediate?)
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout k.pem -out csr.pem \
-subj "/CN=orders" -addext "subjectAltName=DNS:orders.prod.internal,URI:spiffe://prod/orders"
keytool -list -v -keystore keystore.p12 -storetype PKCS12      # what the JVM thinks it has
keytool -importcert -alias int-2026 -file int.pem -cacerts     # add a trust anchor to the JVM default
store

```

Part B — KMS vs. PKI vs. Secrets

B.1 Taxonomy: four different services people lump together

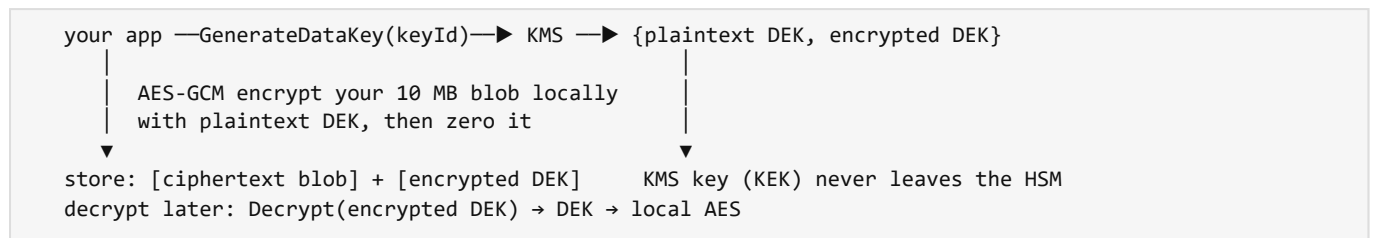
	KMS / Vault (AWS KMS, OCI Vault)	Private CA (AWS Private CA, OCI Certs)	Secrets Manager (AWS Secrets Mgr, OCI Vault secrets)	Wallet / Keystore (Oracle Wallet, JKS, PKCS#12)
Holds:	symmetric/asym KEYS (never leave HSM)	CA keys + issued CERTIFICATES	passwords, API tokens, DB creds	client-side file of certs + keys
Verb:	encrypt/decrypt sign/verify	issue/renew/ revoke cert	get/rotate value	open, read
Rotation:	key material, old versions kept for decrypt	cert renewal, CA rotation	credential value (e.g. Lambda rotator)	manual replace

- Intuition: KMS answers "encrypt/sign this for me and never show me the key".
A CA answers "vouch that this public key belongs to this name". Secrets Manager answers "give me the password".
→ They compose: Secrets Manager encrypts secrets *with* a KMS key; Private CA stores its CA private key *in* an HSM
like KMS does; OCI Certificates uses OCI Vault-backed keys for its CAs.
- Defeater: KMS is the wrong tool for high-volume symmetric encryption of every row.
→ Each `Encrypt` is a network call to an HSM-backed fleet with per-account request quotas

(order of magnitude: thousands of req/s per region, check current quota page).

→ That is why **envelope encryption** exists (B.2).

B.2 AWS KMS features



Feature	What it is	Deciding variable / defeater
KMS key (formerly CMK)	Logical key: ARN + policy + metadata + one or more versions of key material inside FIPS-validated HSMs	Key material for <code>Origin=AWS_KMS</code> is never exportable. Imported material (BYOK) and custom key stores (CloudHSM) change the rotation story.
Symmetric (AES-256-GCM)	Encrypt / Decrypt / GenerateDataKey / ReEncrypt	Payload limit 4 KB per direct Encrypt → forces envelope encryption for anything bigger
Asymmetric (RSA / ECC / SM2)	Sign / Verify or Encrypt / Decrypt with a public key you can export via GetPublicKey	Public key export lets <i>offline</i> verifiers work; private half still in HSM
HMAC keys	GenerateMac / VerifyMac	For tokens where you want KMS-held secret without asymmetric cost
Envelope encryption	KMS wraps a per-object data key (DEK); you encrypt locally	Rule: 1 KMS call per object, not per byte. Failure prevented: KMS throttling + latency on the hot path
Encryption context	Key/value AAD bound into the ciphertext; must match on decrypt	Cheap tenant isolation: <code>{"tenant": "acme"}</code> — decrypt with the wrong context fails. Not secret; logged in CloudTrail
Key policy + IAM policy + grants	Three authorization layers (B.4)	Key policy is the root of trust; an IAM policy alone cannot grant access unless the key policy delegates to IAM
Multi-Region keys	Same key material replicated so ciphertext decrypts in another region	Replicas are the same key; not a DR <i>policy</i> , just a mechanism
Aliases	<code>alias/orders-prod</code> → key ARN, re-pointable	Callers reference alias, you swap the key underneath for manual rotation
CloudTrail	Every API call logged with principal, key, encryption context	The audit story; also how you detect a leaked role hammering Decrypt

- Best practice: one KMS key per data classification × environment, not per table.
→ Failure prevented: policy sprawl; blast radius on a key-policy typo becomes "all prod data".
- Best practice: always pass an encryption context and enforce it with `kms:EncryptionContext:` conditions in policy.
→ Failure prevented: ciphertext from tenant A decrypted under a role scoped to tenant B.

Minimal Java 17 envelope encryption (SDK v2):

```
// 1. Ask KMS for a data key wrapped by the KEK
GenerateDataKeyResponse dk = kms.generateDataKey(r -> r
    .keyId("alias/orders-prod")
    .keySpec(DataKeySpec.AES_256)
    .encryptionContext(Map.of("tenant", tenantId)));

// 2. Encrypt locally with the plaintext DEK (AES-GCM), then discard it
SecretKey dek = new SecretKeySpec(dk.plaintext().asByteArray(), "AES");
Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
c.init(Cipher.ENCRYPT_MODE, dek, new GCMParameterSpec(128, iv));
byte[] ciphertext = c.doFinal(payload);

// 3. Persist ciphertext + dk.ciphertextBlob() (the wrapped DEK) + iv
```

```
// Decrypt path: unwrap DEK (KMS enforces policy + encryption context here), then local AES
DecryptResponse d = kms.decrypt(r -> r
    .ciphertextBlob(wrappedDek)
    .encryptionContext(Map.of("tenant", tenantId))); // mismatch → InvalidCiphertextException
```

B.3 Rotation — what actually rotates

```
KMS key (ARN, policy, alias) — stays the same → callers never change
├ key material v1 (2025-01)
├ key material v2 (2026-01)
└ key material v3 (2027-01) } all kept; ciphertext header says which version to use
                             } new encrypts use newest; decrypts pick the right one
```

- Intuition: automatic rotation replaces the *material* behind a stable handle; nothing you stored becomes unreadable.
 - (AWS KMS docs, 2026-09) Key ID, ARN, region, policies and permissions do not change on rotation, so applications and aliases referencing the key need no change.
 - (AWS KMS docs, 2026-09) Default: material is rotated one year from enabling and every year after; `RotationPeriodInDays` overrides that. (AWS KMS docs, 2026-09) Valid range 90 to 2560 days.
 - (AWS KMS docs, 2026-09) Only symmetric encryption keys with KMS-generated material rotate automatically — not asymmetric, HMAC, imported-material, or custom-key-store keys; those you rotate manually.
 - (AWS KMS docs, 2026-09) Rotation is observable in CloudTrail/CloudWatch, `GetKeyRotationStatus` shows in-progress, `ListKeyRotations` shows history.
- Defeater: rotation does **not** re-encrypt existing data.
 - Old ciphertext stays under old material forever. If you need "data encrypted under v1 must not exist after 2027", you run a re-encrypt job (`ReEncrypt` for wrapped DEKs, or full re-write for envelope data).
 - Manual rotation = create new key, re-point alias. That one *does* change the key ARN under the alias; keep the old key enabled for decrypts.
- Certificate rotation is a different animal (Part C): the *cert* expires on a schedule, so renewal is mandatory, and clients must pick up the new cert before the old one's `notAfter`.

B.4 Access: role → authenticated → authorized → key used

```
[control plane] IAM: role "orders-svc" with trust policy (who may assume) + permission policy (what it may do)
[control plane] KMS key policy on alias/orders-prod: "Allow arn:...:role/orders-svc kms:Decrypt, kms:GenerateDataKey"
                                                         + Condition kms:ViaService /

kms:EncryptionContext:tenant
[hot path]      ECS task —(1) task role creds from metadata endpoint→ STS
                  —(2) SigV4-signed KMS request→ KMS —(3) authz eval→ HSM op
```

Numbered sequence — how "getting a role" turns into "the key was used":

1. **Deployer** attaches an IAM role to the ECS task / Lambda / EC2 instance profile — on deploy. State changed: task has a role ARN. Fails → task starts with no creds; every AWS call 403s.
 2. **Runtime agent** (ECS agent) calls STS `AssumeRole` on the task's behalf and exposes short-lived creds (access key, secret, session token; order of magnitude: hours) at the container credentials endpoint — after 1. Fails → SDK credential chain throws `SdkClientException`; retry with backoff.
 3. **Your SDK** signs the KMS request with SigV4 (HMAC over the canonical request using the secret key) — per call. This is **authentication**: KMS recomputes the signature, so it knows *which principal* sent it. Fails (clock skew > 5 min, bad key) → `InvalidSignatureException`; SDK v2 retries once with the server's clock offset, beyond that fix NTP.
 4. **KMS authorization engine** evaluates, in order: any explicit deny anywhere → deny; then does the **key policy** allow this principal (directly, or by delegating to IAM via the "AWS": "arn:...:root" statement)? then does an **IAM policy** on the principal allow `kms:Decrypt` on this key ARN? then any **grant**? plus condition keys (`kms:ViaService`, `kms:EncryptionContext:*`, `aws:SourceVpce`). Fails → `AccessDeniedException` (terminal; do not retry).
 5. **HSM fleet** performs the op under the current key material version; ciphertext header identifies the version — after 4. Fails → `KMSInternalException` / `DependencyTimeoutException` (retryable, backoff + jitter), `DisabledException` / `KMSInvalidStateException` (terminal — key disabled or pending deletion).
 6. **CloudTrail** records {principal, key ARN, action, encryption context, source IP} — async, after 5. Fails → you lose audit, not availability.
- Defeater: "I gave the role `kms:*` in IAM and it still 403s."
→ The key policy did not delegate to IAM. Key policy is the root; IAM is only consulted if the key policy says so.
 - "Authentication is SigV4 — KMS knows who you are because only you could produce that signature. Authorization is key policy first, IAM second, grants third, explicit deny always wins."

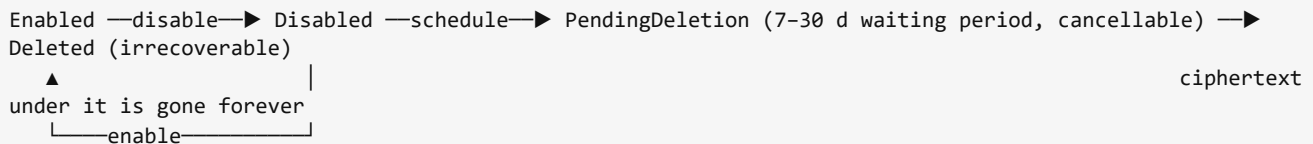
B.5 OCI equivalents (Vault, Certificates, IAM)

From general knowledge; verify current names on docs.oracle.com before quoting in a loop.

AWS	OCI	Note
KMS key	OCI Vault → Master Encryption Key (HSM- or software-protected)	Vault = the container; keys have versions like KMS material versions
Key policy + IAM policy	IAM policy statements on compartments: Allow <code>dynamic-group orders-svc</code> to use keys in compartment prod where <code>target.key.id = '...'</code>	No per-key policy document; authorization is compartment-scoped policy language
IAM role on ECS task	Instance principal / resource principal via a dynamic group	Same idea: workload identity, short-lived token, no static creds
Envelope encryption	Same: <code>GenerateDataEncryptionKey</code> then local AES	
AWS Private CA / ACM	OCI Certificates service: CA hierarchy, cert issuance, auto-renewal, Vault-backed CA keys	Load balancers can pull certs directly by OCID
Secrets Manager	OCI Vault Secrets with versions and rotation	
CloudTrail	OCI Audit	

- If asked "how does a service get a key in OCI" the sequence is the same shape as B.4:
→ compute instance / function ∈ dynamic group → obtains resource-principal token from the instance metadata service
→ signs the request → Vault evaluates policy on the target compartment/key → HSM op → Audit event.

B.6 KMS extras: key states, grants, DEK caching, sign for a CA



- Key states: disable first, watch CloudTrail for `Decrypt` attempts against the disabled key for the whole waiting period, only then schedule deletion.
→ Failure prevented: deleting a key some cold backup still needs; there is no undo after the waiting period.
- Grants: programmatic, temporary permissions on one key for one principal without editing the key policy; how AWS services (EBS, RDS) use your key on your behalf.
→ Defeater: grants are eventually consistent → use the returned grant token on the immediate call, or you get a spurious `AccessDenied`.
- Data-key caching (AWS Encryption SDK): reuse one DEK for N messages / T seconds / B bytes.
→ Deciding variable: KMS quota vs per-object isolation — a cached DEK widens the blast radius of one leaked key from one object to N.
- Sign for a CA (what Part C calls at step 7):
→ `Sign(KeyId, Message=SHA-256(tbsCertificate), MessageType=DIGEST, SigningAlgorithm=ECDSA_SHA_256)`.
→ RAW messages are capped at 4096 bytes and most certs are bigger anyway → always hash locally, send the 32-byte digest. KMS never sees the certificate.
→ Response is the DER-encoded (r, s) for ECDSA / PKCS#1 v1.5 or PSS for RSA — exactly what the X.509 `signatureValue` field expects, no re-encoding.
→ Verifiers never call KMS: they use the public key from the CA certificate (`GetPublicKey` once, at CA creation).
- Quotas (order of magnitude, check the current quota page): symmetric ops thousands/s per account-region; asymmetric Sign **hundreds/s** — that number is the ceiling on issuance in C.1, and why deploy storms need jitter.

Part C — Design doc: internal Certificate Authority service

C.0 One-page design doc

Problem:	~2,000 internal services (assume) need mTLS between them + TLS to DBs. Today: hand-made certs, 1-year lifetimes, expiry outages, no revocation.
For whom:	platform engineers (CA admins), service owners (cert consumers), security (audit).
Success:	0 expiry-caused outages; p95 issue latency < 2 s (assume); 100% of issued certs traceable to a workload identity in audit.
Non-goals:	public/WebPKI trust (buy: ACM / OCI Certs / Let's Encrypt); code-signing; user client certs (phase 2);
Approach:	HSM procurement (use cloud KMS as the HSM). Root CA offline in KMS asymmetric key; one online Intermediate CA per environment; a stateless Issuer API that validates policy, signs CSRs with the intermediate key held in KMS, records the cert; short-lived leaf certs (24 h, assume) so revocation is mostly "stop renewing".
Build vs buy:	BUY the HSM + audit (KMS/Vault). BUILD only policy + issuance + inventory, and only because the policy ("this SPIFFE ID may get this SAN") is org-specific. If OCI Certificates / AWS Private CA
Private CA	policy hooks are enough → don't build. (Interview answer: name this, then proceed.)

C.1 Requirements

Functional (MVP0/MVP1)

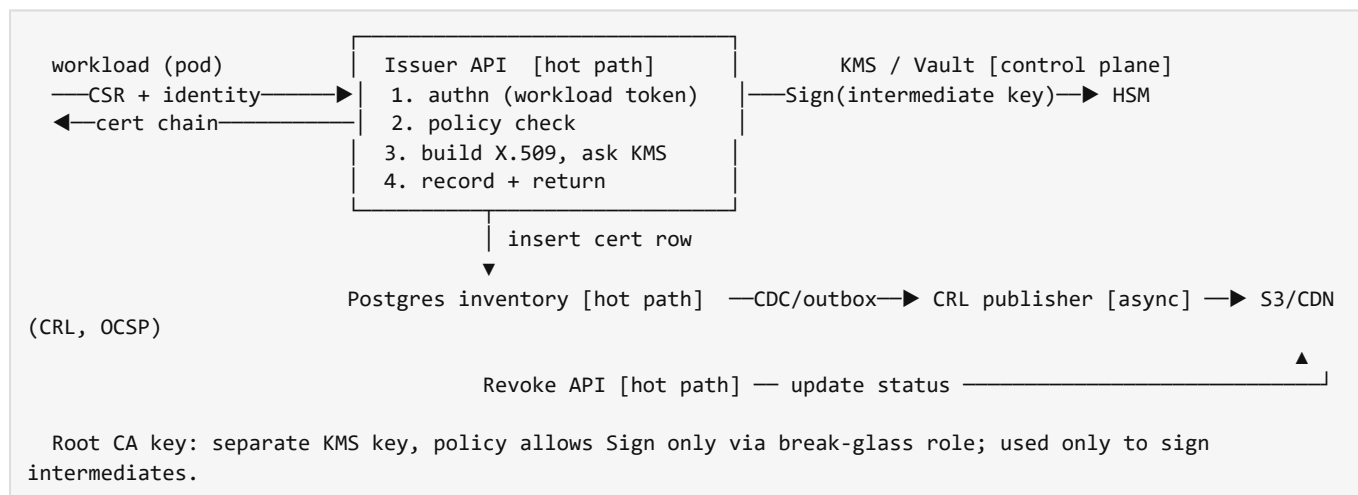
- Issue a leaf cert from a CSR for an authenticated workload, subject/SANs constrained by policy.
- Renew (same as issue with a new key; no "extend" operation).
- Revoke a cert by serial; publish CRL; answer OCSP (MVP1).
- List/inspect certs by owner, expiry, status (inventory).

- Rotate the intermediate CA without breaking verifiers (overlapping validity).
- Extended (defer): user client certs, code signing, external ACME protocol, HSM key ceremony tooling.

Non-functional

Requirement	Target	Why	Enforcing component
Issue latency	p95 < 2 s (assume)	Issuance sits in pod startup path	Issuer API + KMS Sign (order of magnitude: tens of ms)
Issue throughput	peak ~50 issues/s (assume: 2,000 svcs × ~10 replicas, 24 h certs, deploy storms)	Determines whether KMS quota is the wall	KMS quota check; issuer is stateless so it scales horizontally
Availability	99.9% monthly for issue; verification must work at 100% <i>without</i> the service (certs are self-contained; CRL/OCSP cached)	Verifiers must not depend on CA uptime	Cert lifetime ≫ CA outage; CRL TTL
Consistency	Revocation: eventual, ≤ CRL publish interval (15 min, assume). Issuance record: strong (Postgres)	"Revoked but still trusted for 15 min" is the accepted risk with short-lived certs	Postgres for inventory; CDN/S3 for CRL
Durability	CA private keys: never on disk, only in KMS (multi-AZ HSM). Inventory DB: multi-AZ, PITR	Losing the intermediate key = re-issue everything	KMS; Aurora
Security	Private key of the <i>leaf</i> never leaves the requester (CSR model). Every issue tied to a workload identity. Root offline	Threat: stolen issuer box mints arbitrary certs → policy + short lifetimes bound damage	CSR-only API, policy engine, KMS key policy allowing Sign only from issuer role
Auditability	100% of issue/revoke in append-only log	Compliance	CloudTrail on KMS + app audit table
Cost	order of magnitude: KMS asymmetric sign requests are priced per 10k; at 50/s ≈ 130 M/month — check pricing, may justify caching or longer lifetimes		

C.2 High-level design



- Components and single responsibility:
 - **Issuer API** [hot path] — validate identity + policy, assemble TBSCertificate, call KMS Sign, persist. Stateless.
 - **Policy engine** [control plane] — config: which identity may request which SANs/key usages/lifetime. Versioned, in Git.
 - **KMS keys** [control plane] — root (RSA-4096 or P-384, assume), intermediate per env. Private material never leaves HSM.
 - **Inventory DB** (Postgres) [hot path] — serial → cert, owner, status. Source of truth for revocation.
 - **CRL/OCSP publisher** [async] — reads revoked serials, signs CRL with intermediate key, pushes to S3/CDN.
 - **Workload identity** [control plane] — reuse the platform's: IAM role / OCI resource principal / SPIFFE SVID.

We do not build identity.

→ **Trust-bundle distribution** [control plane] — GET /v1/ca/chain + the bundle baked into base images + a node agent that refreshes hourly and reports the bundle hash (the "≥ 95% of verifiers" metric in C.5 rotation). SPIFFE calls this the trust bundle endpoint. Without it, rotation is a guess.

- Where the consistency boundary sits: the Postgres insert is the commit point. A cert that KMS signed but the DB never recorded is an unrevocable orphan → sequence in C.5 orders DB before return and handles the gap.
- Single point of contention: the intermediate KMS key (all signs serialize there). Mitigation: per-env keys, KMS quota raise, issuance cache for identical CSR retries (idempotency).

C.3 APIs

Auth on every call: workload token (IAM SigV4 / OCI resource principal / SPIFFE JWT) at the gateway. Human admin calls: OIDC + admin role.

```
POST /v1/certificates
  Idempotency-Key: <uuid>
  { "csr_pem": "...", "profile": "server-mtls", "ttl_seconds": 86400 }
→ 201 { "serial": "0x1a2b...", "certificate_pem": "...", "chain_pem": "...", "not_after": "2026-09-04T10:00:00Z" }
400 policy violation (SAN not allowed for this identity) – terminal
429 quota – retryable with backoff
503 KMS unavailable – retryable; safe because idempotency key dedups

GET /v1/certificates/{serial}          → cert + status + owner
GET /v1/certificates?owner=&expires_before=&status=&cursor= → paginated inventory
POST /v1/certificates/{serial}:revoke { "reason": "keyCompromise" } → 202 (revocation is eventual until CRL publish)
GET /v1/ca/chain                      → current intermediate + root (public)
GET /crl/{issuer}.crl (S3/CDN, public, signed)
POST /ocsp (MVP1)
POST /v1/ca/intermediates:rotate      admin only; creates new intermediate, both valid for overlap window
```

- Design choice: **CSR-in, never key-out**. Alternative: server generates key pair and returns PKCS#12. → Chosen because the private key then only ever exists on the requester; the CA cannot leak what it never saw. → Flips if consumers are legacy appliances that cannot generate keys (then: generate, return once, never store).
- Idempotency key on issue: a retried issue after a 503 must not mint a second cert with a new serial.
- What a **profile** is (the thing the policy engine evaluates; versioned in Git, CI-tested with allow/deny CSR fixtures):

```
profiles:
  server-mtls:
    key_usage: [digitalSignature]
    ext_key_usage: [serverAuth, clientAuth] # both, or the service fails as a client one hop away
    max_ttl: 24h
    algorithms: [ecdsa-p256, rsa-2048] # rsa-2048 only for legacy (Oracle wallet) consumers
    basic_constraints: "CA:FALSE"
    san_rules: # identity → allowed names; templates over the identity
      "spiffe://prod/{svc}": [{"svc}.prod.internal", "spiffe://prod/{svc}"]
      "arn:aws:iam::123456789012:role/orders-svc": ["orders.prod.internal"]
```

→ Policy = (identity → allowed SANs) × (profile → usages, TTL cap, algorithms). Two tables, one deny wins.

→ Multi-tenant (JD keyword): tenant is part of the identity (spiffe://prod/tenantA/..., OCI compartment), and each tenant's intermediate carries **Name Constraints** for its subtree (A.5) so a policy bug in tenant A's rules still cannot produce a cert for tenant B's names.

C.4 Data model

```
CREATE TABLE certificates (  
  serial          NUMERIC(40) PRIMARY KEY,          -- 128-bit CSPRNG serial (CA/B BR require ≥ 64 bits  
entropy), never sequential  
  issuer_id       TEXT NOT NULL,                    -- which intermediate  
  owner_identity  TEXT NOT NULL,                    -- IAM role ARN / SPIFFE ID  
  profile         TEXT NOT NULL,  
  subject_cn      TEXT NOT NULL,  
  sans            TEXT[] NOT NULL,  
  not_before      TIMESTAMPTZ NOT NULL,  
  not_after       TIMESTAMPTZ NOT NULL,  
  status          TEXT NOT NULL CHECK (status IN ('valid','revoked','expired')),  
  revoked_at      TIMESTAMPTZ,  
  revocation_reason TEXT,  
  cert_pem        TEXT NOT NULL,                    -- no private key column. Ever.  
  idempotency_key UUID UNIQUE,  
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX ON certificates (owner_identity, not_after);  
CREATE INDEX ON certificates (issuer_id, status) WHERE status = 'revoked'; -- CRL build  
  
CREATE TABLE issuers (  
  issuer_id       TEXT PRIMARY KEY,  
  kms_key_arn     TEXT NOT NULL,  
  cert_pem        TEXT NOT NULL,  
  not_after       TIMESTAMPTZ NOT NULL,  
  state           TEXT NOT NULL CHECK (state IN ('active','overlap','retired'))  
);
```

- Store choice: Postgres. Access patterns are by serial, by owner+expiry, by issuer+status — all indexable, low write rate (50/s).
→ NoSQL would flip only if issuance rate were order of magnitude: 10k+/s (e.g., per-request certs), not here.
- Partitioning: none needed at this scale; `certificates` grows $\sim 50/s \times 86,400 \approx 4.3$ M rows/day (assume) → purge expired rows older than retention (90 days, assume) with a partitioned-by-month table if that becomes painful.
- PII: subject CN/SANs are service names, not people. If user certs arrive in phase 2, subject becomes PII → retention policy.

C.5 Core flows (numbered sequences)

Issue

1. **Workload** generates a key pair locally, builds a CSR (CN=orders, SAN=spiffe://prod/orders), signs it with its private key — at startup. Fails → nothing sent.
2. **Gateway** authenticates the workload token (SigV4 / resource principal / SVID) and injects X-Identity — on request. Fails → 401, terminal.
3. **Issuer API** verifies the CSR's self-signature (proof of key possession) — after 2. Fails → 400, terminal (prevents issuing a cert for a public key the caller does not hold).
4. **Issuer API** loads policy for (identity, profile) and checks every SAN, key usage, and TTL ≤ profile max — after 3. Fails → 400 with the violated rule; audit-log the denial.
5. **Issuer API** checks `idempotency_key` in Postgres — after 4. Hit → return the stored cert (200). Miss → continue.
6. **Issuer API** assembles the TBSCertificate (random 128-bit serial, `notBefore = now - 5 min` clock-skew slack, `notAfter = now + ttl`, AKI/SKI, extensions) and computes its DER — after 5. Fails → 500, retryable.
7. **Issuer API** calls KMS `Sign(keyId=intermediate, message=SHA-256(TBS), messageType=DIGEST, algorithm=ECDSA_SHA_256)` — sync, awaits. Fails → 503 to caller, retryable; no state changed yet. (Poison-pill risk: none, request is pure.)
8. **Issuer API** inserts the row in Postgres with `status='valid'` — after 7, awaits commit. Fails → **the dangerous gap**: a signed cert exists that we cannot revoke by serial lookup. Mitigation: do not return the cert; log the serial to an outbox file/queue for reconciliation; return 503 so the caller retries with the same idempotency key

(step 5 misses → we mint a *new* serial; the orphan is unrevocable but expires in ≤ 24 h — this is the argument for short lifetimes).

9. **Issuer API** returns 201 with cert + chain — after 8 commit ack.

10. **Audit sink** receives the event — async, after 8, via CDC/outbox. Fails → alert on lag; issuance unaffected.

Revoke

1. **Admin/owner** calls `:revoke` — on incident. Authz: owner of the cert or CA admin. Fails → 403.

2. **Issuer API** updates `status='revoked', revoked_at=now()` — sync. Fails → 500, retryable (idempotent update).

3. **API** returns 202 — after 2. Clients still accept the cert until CRL refresh; state the window.

4. **CRL publisher** (every 15 min or on outbox event) selects revoked serials for the issuer, builds the CRL, KMS-signs it, writes to S3 with `Cache-Control: max-age=900` — async. Fails → old CRL keeps being served; alert if CRL `nextUpdate` is within $2\times$ the publish interval (verifiers may hard-fail on a stale CRL).

Rotate intermediate CA

1. **Admin** creates a new KMS asymmetric key, generates a CSR for `Intermediate-2027` — planned, quarterly (assume).

2. **Break-glass role** uses the **root** KMS key to sign it (two-person approval on the role assumption) — after 1. Fails → nothing issued.

3. **Admin** inserts `issuers` row with `state='overlap'` and pushes the new chain to `/v1/ca/chain` and to trust-store distribution — after 2.

4. **Issuer API** starts signing with the new key once $\geq 95\%$ of verifiers have reported the new chain (metric from trust-store agent) — after 3. Skipping this step is the classic rotation outage: new certs rejected by old verifiers.

5. **Old intermediate** stays `overlap` until its last issued leaf expires (max TTL = 24 h), then `retired`; its KMS key stays enabled for CRL signing until its own `notAfter`.

Renew (consumer side — the client library, the half of rotation the CA cannot do)

1. **Library** computes `renew_at = notBefore + 50% \times lifetime + jitter(0-10\%)` — at cert load. The CA can be down for half a TTL before any consumer expires; jitter breaks fleet lockstep.

2. **Timer** fires → generate a **new** key pair (never reuse; a cert rotation is also a key rotation) → CSR → POST `/v1/certificates` with a fresh idempotency key — retry with exponential backoff until 90% of lifetime, then page.

3. **Swap in memory, atomically.** Java: build a new `KeyManager/SSLContext` and swap an `AtomicReference` that a delegating `X509ExtendedKeyManager` reads per handshake; Envoy: SDS push; Nginx: `nginx -s reload`. Existing TLS sessions keep the old cert until they close; new handshakes use the new one — no restart, no dropped connections.

4. **Storage:** key + cert in memory or tmpfs, mode 0600, never on a shared volume; the private key has now existed in exactly one process.

5. Fails → step 2 backoff; `certs_expiring_within_1h` (C.6) is the alarm that this loop is broken somewhere in the fleet.

Rotate the root (years; cross-signing)

1. **Ceremony** generates root R2 in a new KMS key (two-person policy, never-used alarm) — planned years ahead of R1 `notAfter`.

2. **Cross-sign:** R1 signs a cert for R2's public key (or for the new intermediate). Same key, two issuer signatures → a chain validates against **either** root. Serve the cross-signed chain.

3. **Distribute R2** to every trust store (base images, wallets, JVM `cacerts`, appliances) over months; track coverage with the trust-bundle agent.

4. When coverage is $\geq 99.x\%$ (assume), drop the cross-cert from the served chain; R1 expires quietly.

◆ Canonical example: Let's Encrypt ISRG Root X1 cross-signed by DST Root CA X3 until Sept 2021; clients that never updated their trust store (old Android, OpenSSL 1.0.2) broke when the old root expired — the defeater is verifiers you do not control.

C.6 Failure modes

Failure	Trigger	Blast radius	Detection signal	Mitigation
Mass expiry outage	CA down > leaf TTL, or renewal agent bug	Every service whose cert expires in the window	<code>certs_expiring_within_1h</code> gauge; renewal error rate	Renew at 50% of TTL (12 h for 24 h certs) → CA can be down 12 h before impact; alert at 70%
KMS throttling	Deploy storm	New issues fail; running services fine	429 rate from KMS in Datadog	Idempotent retries + jitter; per-env keys; raise quota; renewal jitter so fleets don't renew in lockstep
Intermediate key compromise	Issuer host / role stolen	Every cert signed since compromise	Anomalous issuance (unknown SANs, off-hours volume) from audit	Revoke intermediate via root CRL; rotate (C.5); short TTL bounds exposure to ≤ 24 h
Root key compromise	KMS account compromise	Everything	Root <code>Sign</code> CloudTrail event outside a ceremony	Root key policy: <code>Sign</code> only from a break-glass role with MFA + approval; alert on <i>any</i> root use
Stale CRL / OCSP down	Publisher failure	Verifiers soft-fail (accept revoked) or hard-fail (reject all) depending on config	CRL <code>nextUpdate - now</code> gauge	Publish interval ≪ CRL validity; serve from CDN; prefer short-lived certs so revocation matters less
Orphan cert (signed, not recorded)	DB failure between steps 7 and 8	One cert, ≤ TTL	Outbox reconciliation finds serial not in DB	Reconciler inserts as <code>revoked</code> ; short TTL
Clock skew	Node NTP drift	"cert not yet valid" at startup	<code>notBefore</code> rejections in client logs	<code>notBefore = now - 5 min</code> ; NTP monitoring
Policy misconfig allows wildcard SAN	Bad policy PR	Any service can impersonate any other	Policy diff CI check; issuance audit	Policy in Git, reviewed; CI test suite of allowed/denied CSR fixtures

- Exception handling summary:
 - Retryable: KMS 5xx / throttling, DB transient, CRL upload. Terminal: policy violation, bad CSR signature, access denied, key disabled.
 - What the service swallows: nothing on the issue path; audit-sink failures are logged and alerted, never block issuance.

C.7 MVP ladder + what we evolve next

- **MVP0** — one intermediate key in KMS, `POST /v1/certificates` with a hard-coded policy map, Postgres inventory, `GET /v1/ca/chain`. No revocation (24 h TTL is the revocation). Demo: pod gets a cert, curl with mTLS works.
- **MVP1** — policy engine from Git config, revoke + CRL publisher to S3, idempotency keys, Datadog dashboards for expiry/throttle, renewal client library (renew at 50% TTL with jitter).
- **MVP2** — OCSP responder, intermediate rotation runbook + trust-store agent, SPIFFE/SPIRE integration so identity is not bespoke, user client certs.

Questions to evolve from (ask me for any of these next):

- Why not just use AWS Private CA / OCI Certificates end-to-end? (Answer: probably should; the build case is only custom policy + inventory UX.)
- How does this change if certs are per-request (service mesh at 10k/s)? (Answer: local issuers per node with delegated intermediates; KMS moves off the hot path.)
- How do you do the root key ceremony? (Offline, split knowledge, or KMS key with two-person policy + never-used alarm.)

- How would you design the KMS itself (the HSM-backed key service), not the CA? (Different problem: multi-tenant key isolation, request signing, key material versioning, regional replication.)

C.8 Key function code: `issue(csr)`

The mechanism an interviewer asks you to write: verify possession → policy → assemble TBS → sign with a key you never hold. Python first (local key, `cryptography`), then Java with the KMS hook. Both blocks were run and verified against the issuing key, including the deny cases.

```
import datetime as dt, secrets
from cryptography import x509
from cryptography.x509.oid import ExtendedKeyUsageOID as EKU
from cryptography.hazmat.primitives import hashes

# policy: identity -> allowed SANs (from Git config); profile -> usages + max TTL
POLICY = {"spiffe://prod/orders": {"orders.prod.internal", "spiffe://prod/orders"}}
PROFILES = {"server-mtls": {"max_ttl": 86400, "eku": [EKU.SERVER_AUTH, EKU.CLIENT_AUTH]}}

def issue(csr_pem: bytes, identity: str, profile: str, ttl: int, issuer_cert, issuer_key):
    csr = x509.load_pem_x509_csr(csr_pem)
    if not csr.is_signature_valid:
        raise ValueError("bad CSR signature")
    prof = PROFILES[profile]
    if ttl > prof["max_ttl"]:
        raise ValueError("ttl over profile max")
    san = csr.extensions.get_extension_for_class(x509.SubjectAlternativeName).value
    names = {n.value for n in san}
    if not names <= POLICY.get(identity, set()):
        raise ValueError(f"SAN not allowed for {identity}: {names}")
    now = dt.datetime.now(dt.timezone.utc)
    b = (x509.CertificateBuilder()
        .subject_name(csr.subject).issuer_name(issuer_cert.subject)
        .public_key(csr.public_key())
        .serial_number(secrets.randbits(127))
        .not_valid_before(now - dt.timedelta(minutes=5))
        .not_valid_after(now + dt.timedelta(seconds=ttl))
        .add_extension(san, critical=False)
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
        .add_extension(x509.KeyUsage(digital_signature=True, key_encipherment=False,
            content_commitment=False, data_encipherment=False, key_agreement=False,
            key_cert_sign=False, crl_sign=False, encipher_only=False, decipher_only=False), critical=True)
        .add_extension(x509.ExtendedKeyUsage(prof["eku"]), critical=False)
        .add_extension(x509.SubjectKeyIdentifier.from_public_key(csr.public_key()), critical=False)
        .add_extension(x509.AuthorityKeyIdentifier.from_issuer_public_key(issuer_cert.public_key()),
            critical=False))
    return b.sign(issuer_key, hashes.SHA256())

# Time: O(#SANs) policy check + one signature; the signature (KMS round-trip) dominates.
# Caveat: `cryptography` needs the private key in-process; for an HSM/KMS-held key use the
# ContentSigner pattern below or PKCS#11. Verify: cert.verify_directly_issued_by(issuer_cert).
```

```

// BouncyCastle (bcprov). The only thing that touches the private key is DigestSigner –
// real impl: kms.sign(r -> r.keyId(arn).message(SdkBytes.fromByteArray(digest))
// .messageType(MessageType.DIGEST).signingAlgorithm(ECDSA_SHA_256)).signature()
interface DigestSigner { byte[] signSha256Digest(byte[] digest); }

/** Buffers DER(tbsCertificate), hashes locally, sends only the 32-byte digest out. */
static ContentSigner kmsContentSigner(DigestSigner kms) {
    ByteArrayOutputStream tbs = new ByteArrayOutputStream();
    return new ContentSigner() {
        public AlgorithmIdentifier getAlgorithmIdentifier() {
            return new DefaultSignatureAlgorithmIdentifierFinder().find("SHA256withECDSA");
        }
        public OutputStream getOutputStream() { return tbs; }
        public byte[] getSignature() {
            try {
                byte[] digest = MessageDigest.getInstance("SHA-256").digest(tbs.toByteArray());
                return kms.signSha256Digest(digest); // KMS returns DER (r,s) – exactly what X.509
expects
            } catch (GeneralSecurityException e) { throw new IllegalStateException(e); }
        }
    };
}

static final Map<String, Set<String>> POLICY =
    Map.of("spiffe://prod/orders", Set.of("orders.prod.internal", "spiffe://prod/orders"));

static X509Certificate issue(PKCS10CertificationRequest csr, String identity, long ttlSec,
    X509Certificate issuerCert, DigestSigner kms) throws Exception {
    // 1. proof of possession: CSR self-signature
    if (!csr.isSignatureValid(new
JcaContentVerifierProviderBuilder().build(csr.getSubjectPublicKeyInfo())))
        throw new SecurityException("bad CSR signature");
    // 2. policy: every requested SAN must be allowed for this identity
    GeneralNames san = GeneralNames.fromExtensions(csr.getRequestedExtensions(),
Extension.subjectAlternativeName);
    for (GeneralName n : san.getNames())
        if (!POLICY.getOrDefault(identity, Set.of()).contains(n.getName().toString()))
            throw new SecurityException("SAN not allowed for " + identity + ": " + n.getName());
    // 3. TBS assembly
    Instant now = Instant.now();
    BigInteger serial = new BigInteger(127, SecureRandom.getInstanceStrong()); // >= 64 bits entropy
(CA/B); 128 here
    JcaX509ExtensionUtils u = new JcaX509ExtensionUtils();
    X509v3CertificateBuilder b = new JcaX509v3CertificateBuilder(
        X500Name.getInstance(issuerCert.getSubjectX500Principal().getEncoded()), serial,
        Date.from(now.minusSeconds(300)), // clock-skew slack
        Date.from(now.plusSeconds(ttlSec)),
        csr.getSubject(), csr.getSubjectPublicKeyInfo())
        .addExtension(Extension.subjectAlternativeName, false, san)
        .addExtension(Extension.basicConstraints, true, new BasicConstraints(false))
        .addExtension(Extension.keyUsage, true, new KeyUsage(KeyUsage.digitalSignature))
        .addExtension(Extension.extendedKeyUsage, false,
            new ExtendedKeyUsage(new KeyPurposeId[]{KeyPurposeId.id_kp_serverAuth,
KeyPurposeId.id_kp_clientAuth}))
        .addExtension(Extension.subjectKeyIdentifier, false,
u.createSubjectKeyIdentifier(csr.getSubjectPublicKeyInfo()))
        .addExtension(Extension.authorityKeyIdentifier, false, u.createAuthorityKeyIdentifier(issuerCert));
    // 4. sign: the builder streams DER(tbs) into our ContentSigner; the private key is never in this JVM
    X509CertificateHolder h = b.build(kmsContentSigner(kms));
    return new JcaX509CertificateConverter().getCertificate(h);
}
// Time: O(#SANs) + one KMS round-trip (tens of ms) – the round-trip dominates.
// Caveat: the builder calls getSignature() once per build; a retry must rebuild (new serial) or hit the
idempotency row first.

```

- "The CA process never holds a private key. It holds a role that may call `sign` on one KMS key, and it sends a digest, not a cert."

→ Test the stand-in: a local EC key behind `DigestSigner` using `NONEwithECDSA` over the digest;
`leaf.verify(caPublicKey)` passes → the KMS path is byte-identical.

C.9 Build vs buy

Option	What it is	Strength	Weakness	Choose when
AWS Private CA	Managed CA hierarchy, HSM keys, CRL/OCSP, templates, ACM + EKS/AD connectors	Zero ops; audit and revocation built in; short-lived-cert mode is cheap	Per-CA monthly fee (order of magnitude hundreds of \$; check pricing); policy = templates + IAM conditions, not arbitrary rules	AWS-native fleet whose policy fits "this role may request this template"
OCI Certificates	CA hierarchy, versioned certs, auto-renewal, Vault-backed keys, LB/API-GW integration	Native compartment authz + Audit; consumers pull by OCID	Fewer protocols (no ACME; verify current CRL/OCSP support); policy expressiveness limited	OCI fleet — the team's own product; say so
HashiCorp Vault PKI	API-driven CA secrets engine: roles = profiles, short-lived certs, HSM/KMS via managed keys	Very flexible roles, multi-cloud, huge ecosystem	You run Vault (HA, unseal, upgrades); BSL license since 2023	Multi-cloud, Vault already in production
smallstep step-ca	OSS CA with ACME, SSH certs, cloud-KMS signers	Small, readable, ACME for internal clients	You operate it; small vendor	Dev/internal PKI where consumers speak ACME
SPIFFE / SPIRE	Workload identity + SVID (X.509) issuance with node/workload attestation, federation, upstream-CA plugin	Solves identity and certs together; rotation built into the agent	Agent on every node; not a general certificate service (no long-lived certs, no wallets)	Service-mesh mTLS at scale; pair with any of the above as upstream CA
cert-manager	Kubernetes controller: issuers (ACME, Vault, Private CA, CA key in a Secret)	K8s-native, declarative	Kubernetes only; CA-key-in-Secret mode is a foot-gun	K8s workloads, with an external issuer behind it
Build (Part C)	Policy + issuance + inventory over KMS-held keys	Org-specific policy, inventory UX, exact control of the audit story	You own the correctness of a security-critical system: ceremony, audits, on-call for expiry	Only when a managed CA's templates cannot express "identity X may get SAN Y"

- Pick: **managed CA (OCI Certificates / AWS Private CA) as the signer + SPIRE for identity; build only the policy/inventory layer on top.**
 → Deciding variable: can the managed product express your issuance rule? If yes, the build case collapses to a thin policy proxy in front of it — which is what Part C's Issuer API becomes.

C.10 ? Pop-up questions

- **? Why does the CA never generate the private key for the requester?**
 → L4: CSR model = proof of possession; the key exists only where it is used, so a CA breach leaks nothing usable. Key-out (PKCS#12) only for appliances that cannot generate keys, returned once and never stored.
 → L5: the CSR signature also stops a "confused deputy" issuing for a public key the caller found elsewhere — validate it before policy, so denials do not leak which names exist.
- **? A Java client fails with "unable to find valid certification path" but Chrome opens the same endpoint. What happened?**
 → L4: the server sends only the leaf; Chrome fetched the intermediate via AIA, Java does not. Fix the chain file (leaf + intermediates), verify with `openssl s_client -showcerts`.
 → L5: second candidate is AKI/SKI mismatch during an intermediate overlap; third is a JVM whose `cacerts` never got the new root — which is why the trust-bundle agent reports coverage.
- **? How do you make revocation actually work?**
 → L4: short-lived leaves so revocation ≈ stop renewing; CRL on a CDN as backstop with `nextUpdate` alarms; OCSP stapling only for public endpoints.

- L5: name the soft-fail dilemma (A.6) and say which verifiers you control; for the ones you do not, the TTL is your revocation SLA.
 - **? Rotate the intermediate with zero outage.**
 - L4: new key + cert, `overlap` state, distribute the bundle first, switch the signer only when coverage $\geq$ threshold, keep the old key for CRL signing until its last leaf expires.
 - L5: the same shape for the root uses cross-signing (C.5); the failure is always a verifier you did not know existed — build the inventory of verifiers before the rotation, not during.
 - **? Where is the consistency boundary in issuance, and what is the orphan-cert problem?**
 - L4: the Postgres insert is the commit; a signed-but-unrecorded cert cannot be revoked by serial. Order: sign → insert → return; on insert failure, do not return the cert, outbox the serial for the reconciler.
 - L5: it is unavoidable without a distributed transaction across KMS and DB; the accepted risk is bounded by TTL and by the reconciler inserting the orphan as `revoked`. Say the bound out loud.
 - **? KMS starts throttling during a deploy storm. What do you do in the next 10 minutes, and next quarter?**
 - L4: now — retries with jitter are already idempotent, so nothing is duplicated; shed by returning 429 with `Retry-After`; per-env keys spread the quota. Quarter — renewal jitter in the client library, raise the quota, longer TTL for the biggest fleets.
 - L5: move KMS off the hot path: per-node or per-cluster delegated issuers with short-lived, name-constrained sub-CAs (what SPIRE and Istio do) — KMS then signs one sub-CA per node per day instead of one leaf per pod.
 - **? Multi-tenant: how do you stop tenant A from obtaining a cert for tenant B's name?**
 - L4: identity carries the tenant; policy maps identity → allowed SANs; every issue is audited with the identity.
 - L5: defense in depth — per-tenant intermediate with Name Constraints, so even a policy bug cannot produce a cross-tenant cert; compartment-scoped authz on the API; anomaly detection on issuance patterns.
 - **? Backward compatibility: an Oracle DB client only trusts RSA and reads a wallet. How do you serve it without weakening the rest?**
 - L4: an `rsa-legacy` profile with RSA-2048, longer TTL, a separate RSA chain, packaged as PKCS#12 for the wallet; CRL published where that client will actually fetch it.
 - L5: keep the legacy chain on its own intermediate so its longer TTLs and weaker revocation never widen the blast radius of the ECDSA fleet; put a sunset date on the profile and count its consumers monthly.
-

🔑 One-liners

- "A certificate is a public key + a name + a CA signature + an expiry. The private key never leaves the holder."
- "KMS is 'use my key without seeing it'; a CA is 'vouch that this key belongs to this name'; Secrets Manager is 'give me the password'."
- "Envelope encryption: one KMS call per object, local AES per byte. Otherwise you pay KMS latency and quota on every row."
- "KMS rotation swaps the material behind a stable ARN and keeps old versions for decrypt — it does not re-encrypt your data."
- "Authn is SigV4: the service knows who you are because only your secret could produce that signature. Authz is key policy first, IAM second, grants third, explicit deny wins."
- "Short-lived certs turn revocation from a distributed-cache-invalidation problem into 'stop renewing'."
- "The dangerous gap in a CA is 'signed but not recorded' — order the DB commit before returning the cert, and let short TTLs bound the orphan."
- "In a CA rotation, the outage is never the new key; it is verifiers that haven't loaded the new chain."
- "The CA process holds no private key — it holds a role that may call `Sign` on one KMS key, and it sends a digest, not a cert."
- "Name Constraints on a per-tenant intermediate are the control that survives a policy bug."

References

1. smallstep/certificates (step-ca) — a real, readable Go CA with KMS/HSM signing backends and ACME: <https://github.com/smallstep/certificates>
2. AWS KMS Developer Guide — key rotation, key policies, envelope encryption: <https://docs.aws.amazon.com/kms/latest/developerguide/rotating-keys-enable.html>
3. Oracle Database 18c Administrator's Reference for Windows — Oracle PKI / Windows PKI integration (the page you linked): <https://docs.oracle.com/en/database/oracle/oracle-database/18/ntqrf/oracle-pki-integration-with-windows.html>
4. SPIFFE/SPIRE — workload identity + short-lived X.509 SVIDs, the modern internal-PKI reference: <https://github.com/spiffe/spire>
5. Fox-IT / Dutch government report on the DigiNotar CA compromise (2011) — the canonical "root compromise" postmortem: <https://www.rijksoverheid.nl/documenten/rapporten/2012/08/13/black-tulip-update>
6. RFC 5280 — X.509 certificate/CRL profile and the path-validation algorithm (A.5): <https://www.rfc-editor.org/rfc/rfc5280>
7. AWS KMS Developer Guide — asymmetric keys, `Sign` with `MessageType=DIGEST`, key states, grants: <https://docs.aws.amazon.com/kms/latest/developerguide/asymmetric-key-specs.html>
8. Let's Encrypt — DST Root CA X3 expiration and the ISRG Root X1 cross-sign (the canonical root-rotation story): <https://letsencrypt.org/docs/dst-root-ca-x3-expiration-september-2021/>
9. CA/Browser Forum Baseline Requirements — serial-number entropy, validity limits, SAN rules: <https://cabforum.org/working-groups/server/baseline-requirements/documents/>